Computer Science 2 Programming Java Style Guide

By Dr. Andrew Steinberg

Latest Revision Spring 2026

1 Introduction

The purpose of this style guide is to help you write clean readable code. While computers can easily compile code, it is important for you and other programmers (especially your course professors and TAs) to read your code properly. You will also encounter times when you need to debug code when logical errors occurs. It is also important to write readable code as your future professors (including me) will expect it. This guide is to help you with common things to avoid when writing code and to avoid losing points on assignments.

2 Curly Braces {}

Any time you open a curly brace, that curly brace should start on a new line. When you write your closing curly brace, that curly brace should also be on a new line and vertically aligned with its counter opening curly brace.

```
1  public class GoodExample
2  {
3      public static void main(String [] args)
4      {
5          int x = 23;
6
7          if(x % 2 == 0)
8          {
9              System.out.println("x is even!");
10         }
11         else
12         {
13             System.out.println("x is odd!");
14         }
15
16         System.out.println("The vertical alignment is nice.");
17     }
18 }
```

Listing 1: Good example where curly braces are on their own line and perfectly aligned with each other. ☺

```java
public class BadExample{
    public static void main(String [] args){
        int x = 23;

        if(x % 2 == 0){
            System.out.println("x is even!");
        }
        else{
            System.out.println("x is odd!");}

        System.out.println("The vertical alignment is nice.");
    }
}
```

Listing 2: Bad example where curly braces are not on their own line and are not perfectly aligned with their counter parts. Points will be deducted if the graders see this in your code files. ☹

3 Comments

Self documentation is very important when writing general code. It serves two big purposes.

1. Allows for you and fellow programmers (professors and TAs) to read English statements in understanding what you have implemented from a high level perspective.

2. You might close a project for some time (months or even years) and then later reopen it to make certain updates. After all the time passing, you might forget your reasons of writing certain statements. This is one of the big reasons commenting is important.

When writing comments, one important rule is keep them simple and to the point. This is to prevent long comments being written that can cause your Java file to expand without wrapping around and becoming unreadable. Consider these examples of what to do and not to do.

```
1  public class BadExample
2  {
3      public static void main(String [] args)
4      {
5          System.out.println("Perform calculation..."); //Display a
       message that tells the user a calculation is about to be
       performed
6
7          //Declaring variables named x and y allows for RAM to
       allocate space for storing two integer type values for some
       calculation
8          int x = 4;
9          int y = 3;
10
11         //Performing calculation on two the variables x and y by
       taking their sum and storing the result in another integer type
        variable called sum
12         int sum = x + y;
13
14         System.out.println("Sum is " + sum); //display result
15     }
16
17 }
```

Listing 3: Bad example where comments are too long. ☹

Listing 3 shows an example of when comments are not simple and to the point. First time programmers make this common mistake and we want to avoid this. If a grader sees this in your programming assignments, points will be deducted in coding style. Listing 4 on the next page shows the same code, but with how the commenting should ideally look.

```java
public class GoodExample
{
    public static void main(String [] args)
    {
        //display message
        System.out.println("Perform calculation...");

        //declare variables and assign values
        int x = 4;
        int y = 3;

        //perform calculation
        int sum = x + y;

        System.out.println("Sum is " + sum); //display result
    }
}
```

Listing 4: Good example where all comments are short and to the point. ☺

Listing 4 shows how comments can be kept simple and to the point. You might notice that some of them are considered grammatical fragments. This is absolutely ok to do in programming. As we evaluate your code throughout the semester, we hope you gain good commenting practice.

3.1 Above-Line and End-Line Comments

Let's first define the difference between above-line and end-line comments.

Above-Line comments are comments that are located directly above the set of code statements it is relative to. The example below shows an above-line comment.

```java
public class AboveLineCommentExample
{
    public static void main(String [] args)
    {
        //this is an above-line comment
        int x = 4;
        int y = 3;
    }
}
```

Listing 5: Above-Line comment example.

End-Line comments are comments that are located directly on the same line as a code statement. Generally it is placed at the end. The comment itself refers to the statement on the same line. The example below shows an end-line comment.

```
public class EndLineCommentExample
{
    public static void main(String [] args)
    {
        int x = 4; //this is an end-line comment
        int y = 3;
    }
}
```

Listing 6: End-Line comment example.

Generally when writing either comment (above or end), it is always important to keep it short and to the point. We are not looking for an essay. Most students ask what is the word limit we can use per comment. The limit is dependent on the readability of the document. For example, if you wrote a 20 word end-line comment, you wouldn't see all the words without horizontally scrolling throughout the document (or zooming out the file). This is generally not good practice. Therefore, here is the recommended limit for end-line comments. The maximum number of words should be no more than 10 words per end line comment. **If we see end-line comments with more than 10 words, the graders will deduct style points. If we see above-line comments with more than 14 words, the graders will deduct style points.** Above-line comments get a few extra words since they are describing a set of statements rather than one. Please also avoid stacking above line comments. Multiline comments should be utilized over stacking single line comments.

3.2 Multiline Comments

The previous subsection showed single line comments, but we also know that multiline comments exist in Java. Multiline comments are generally used if it is hard to compress an above-line comment within its maximum word limit on a single line. If a student plans to use the multiline comment, they must use it in the form of an above-line comment with a max of 14 words per line. The following shows an example of writing a good style multiline comment with the rule of 14 words per line. Note: Multiline comments should NEVER be used as end-line comments. If we find multiline comments used as end-line comments in your file, the graders will take off points in coding style.

```
/*
   This is a comment about maintaing good style for writing
   multiline comments.
*/
```

Listing 7: Example of writing a good style multiline comment.

4 Variable Naming

Throughout the course, variables will be useful for storing values. It's imperative that the names given to the variables have meaning and what their purpose is in a program. Giving variables random names that have no correlation to the program purpose will cause confusion for the readers (especially your graders). Make sure to give proper names to your variables!

```
public class BadExample
{
    public static void main(String [] args)
    {
        System.out.println("Perform calculation...");

        //declare variables and assign values
        int pancakes = 4;
        int gryffindor = 3;

        //perform calculation
        int football = pancakes + gryffindor;

        System.out.println("Sum is " + football);
    }
}
```

Listing 8: Bad example where variable names are not closely related to their functionality and purpose in the program. ☹

```java
public class GoodExample
{
    System.out.println("Perform calculation..."); //display message

    //declare variables and assign values
    int val1 = 4;
    int val2 = 3;

    //perform calculation
    int sum = val1 + val2;

    System.out.println("Sum is " + sum); //display result

}
```

Listing 9: Good example where all variables have proper names. ☺

4.1 Camel Casing Naming Convention

Students are expected to follow good practices in the format of their variable naming. Students must use **camel casing**. If the graders do not see camel casing applied to variable naming, points will be deducted on your assignments.

Camel casing is the naming convention where an uppercase letter is used to represent sequences of words in a variable name. The uppercase letter is only for second word and beyond. For example, if we wanted to name a variable `numerator calculation` using camel casing, we would get

```
int numeratorCalculation;
```

5 Class Naming

Students are expected to use Pascal Casing when defining classes in their code. Pascal casing is where the first letter of each word in the name is capitalized (uppercase). Please take a look at some previous coding examples of class names as they utilize Pascal casing. If students do not follow this, then they will have points deducted from their assignments.

6 Consistent Horizontal Indentation

For those that took COP2500 and learned Python, you might remember that Python is picky with consistent horizontal indentation of code statements from its syntax. While the Java Language does not have this syntax rule, we are still going to follow consistent horizontal indentation of code statements. The following 2 listings here shows inconsistent and consistent indentation.

```
1  public class BadExample
2  {
3      public static void main(String [] args)
4      {
5          System.out.println("Hi There!");
6    System.out.println("This statement is aligned differently!");
7
8      int var = 12;
9  System.out.println("Ending program now!");
10     }
11 }
```

Listing 10: Inconsistent indentation. ☹

Listing 10 shows an example where each statement is not properly consistent with its horizontal indentation. This is bad in practice and the graders will deduct points if they see this in your programming assignments.

```
1  public class GoodExample
2  {
3      public static void main(String [] args)
4      {
5          System.out.println("Hi There!");
6          System.out.println("This statement is aligned perfectly!");
7
8          int var = 12;
9
10         System.out.println("Ending program now!");
11     }
12 }
```

Listing 11: Consistent indentation. ☺

Listing 11 shows an example where each statement is properly consistent with its horizontal indentation. This is what we want to see throughout your programming assignments during the semester.

6.1 Control Structures and Consistent Indentation

Throughout the semester, your programming assignments in Java will require the use of control structures (part of the Java syntax). It is expected that the horizontal indentation is consistent with respect to its control structures. The following two examples show consistent indentation when multiple control structures are used.

```
1  public class BadExample
2  {
3      int x = 1;
4      int y = 4;
5
6      if(x > y)
7      {
8          System.out.println("x is greater than y.\n");
9          System.out.println("The condition evaluated true!\n");
10     }
11     else
12     {
13 System.out.println("x is less than y.\n");
14 System.out.println("The condition evaluated false!\n");
15 }
16 }
```

Listing 12: Bad indentation involving control structures. ☹

Listing 12 shows two problems. The first problem is the indentation of the two print statements on lines 13 and 14 are not properly indented within the else's control structure. They should follow the exact pattern as the print statements from lines 8 and 9.

The second problem relates to the control structures that are associated with the else statement (lines 12 and 15). They are not aligned with each other. This is problematic from a programmer's perspective. This can lead to variable scope confusion along with other misunderstandings. It is good practice to have respective control structures align with each other (both horizontally and vertically). If you notice lines 7 and 10, we can conclude that any statements between are associated within that group of control structures. This is what programmers want to see when evaluating ones code. If the graders see anything like lines 12-15 will lose points in coding style. Listing 13 on the next page shows good indentation involving multiple control structures. The graders will be looking for this kind of style throughout all of your assignments.

```java
public class BadExample
{
    int x = 1;
    int y = 4;

    if(x > y)
    {
        System.out.println("x is greater than y.\n");
        System.out.println("The condition evaluated true!\n");
    }
    else
    {
        System.out.println("x is less than y.\n");
        System.out.println("The condition evaluated false!\n");
    }
}
```

Listing 13: Good indentation involving control structures. ☺

7 Vertical Spacing of Statements

When writing code, you want to make sure you space it out nicely without it looking crammed and hard to read. My suggestion is to group statements that work together on each line sequentially and then add an extra vertical space for the next group of statements. Please also do not put a lot of unnecessary vertical spacing as that is considered bad style. Please look at the following pictures for clarifications.

```java
public class BadExample
{
    public static void main(String [] args)
    {
        System.out.println("Perform calculation..."); //display
        //declare variables and assign values
        int x = 4;
        int y = 3;
        //perform calculation
        int sum = x + y;
        System.out.println("Sum is " + sum); //display result
    }
}
```

Listing 14: Bad example where all statements are crammed. ☹

The listing in 14 shows a simple Java file where all lines contain some sort of statement. This is considered bad practice. Please make sure to space your statements properly and consistently. Graders will take off points if code is submitted like this.

```
public class BadExample
{
    public static void main(String [] args)
    {
        System.out.println("Perform calculation..."); //display



        //declare variables and assign values




        int x = 4;
        int y = 3;




        //perform calculation
        int sum = x + y;




        System.out.println("Sum is " + sum); //display result









    }
}
```

Listing 15: Bad example where all statements are spaced out too much! ☹

The listing in 15 shows a sample file that has two problems. One is inconsistent vertical spacing. The other is there is too much white vertical space in this file. While it is good space out grouped statements, you need to keep it consistent. Inconsistent spacing and too much unnecessary spacing will result in point deductions. Checkout the next example on the next page about good consistent spacing of statements.

```java
public class GoodExample
{
    public static void main(String [] args)
    {
        System.out.println("Perform calculation...\n"); //display

        //declare variables and assign values
        int x = 4;
        int y = 3;

        //perform calculation
        int sum = x + y;

        System.out.println("Sum is %d\n", sum); //display result
    }
}
```

Listing 16: Good example where all statements are spaced nicely. ☺

Listing 17 shows a perfect example of how Java code should be spaced in a file. This readable and will make all your graders and professors very happy! Please space your code like this!!

8 Line Stacking

Line stacking in code refers to putting multiple statements or operations on a single line instead of spreading them across multiple lines. Line stacking should not be seen your programming assignments. If line stacking is detected, then points will be deducted. The following shows an example of line stacking.

```java
int x = 5; int y = 10; int z = x + y; System.out.println(z);
if (x > 0) y++; z++; System.out.println("Done");
```

Listing 17: Line stacking multiple statements on one line.